

Энергоэффективный умножитель Бута
с вынесенной перекодировкой
для адаптивных цифровых фильтров
Выпускная квалификационная работа бакалавра

Сажнев М. Е.

Московский физико-технический институт

МФТИ, 2026

- Умножитель — доминирующий блок по площади и энергопотреблению в DSP
- В адаптивных фильтрах работает непрерывно $\Rightarrow$ определяет энергобюджет
- Многие DSP-задачи устойчивы к погрешности $\Rightarrow$ приближённые вычисления:
доли % точности $\longleftrightarrow$ осязаемая экономия энергии
- Базовый приём — кодирование Бута radix-4 (вдвое меньше частичных произведений)

Ключевой вопрос

Как применить recoding с zero-skipping (Университет Бэйхан, 2025 [?]) в реалистичном сценарии адаптивного фильтра?

Цель работы: снизить энергопотребление умножителя Бута в адаптивном цифровом фильтре за счёт перекодировки с пропуском нулевых кодов.

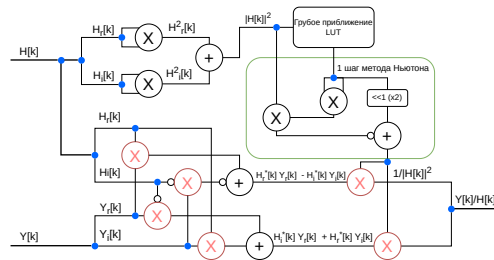
Задачи:

1. Адаптировать алгоритм recoding (Университет Бэйхан [?]) к параллельной архитектуре умножителя
2. Предложить архитектуру с **вынесённым наружу** перекодировщиком (амортизация по тапам фильтра)
3. Реализовать на Verilog три варианта: naive, booth, booth_extrec
4. Построить методику измерения площади и мощности (открытый маршрут синтеза, 45 нм)
5. Оценить точность приближённых режимов (NMED, MRED, SNR) на статистике 10^6 пар

Умножитель Бута radix-4 в составе N -тапового адаптивного фильтра

Принцип radix-4 Booth:

- Множитель — перекрывающиеся группы по 3 бита
- Каждая группа $\rightarrow$ код из набора $\{\bar{2}, \bar{1}, 0, 1, 2\}$
- Вдвое меньше частичных произведений



Ключевая асимметрия

Коэффициенты обновляются **редко** (раз в $10-10^3$ тактов), поток отсчётов — **каждый такт**

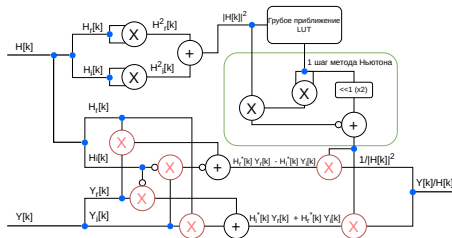
Метод: перекодировка с пропуском нулей

Идея (Университет Бэйхан [?]): пара соседних ненулевых кодов Бута переписывается в эквивалентную пару, **содержащую ноль**:

$$K_{2i+1} = M_{2i+1} - \overline{M_{2i+1}}, \quad K_{2i} = M_{2i} + 4M_{2i+1}$$

- Точно (ER): паттерны 00100, 11011, ошибка $ED = 0$
- Приблизённо (AR): ещё 4 паттерна (00101, 00110, 11001, 11010), $ED = \pm 1$

Больше нулевых кодов $\Rightarrow$ больше занулённых частичных произведений.



Зануление строк в матрице частичных произведений

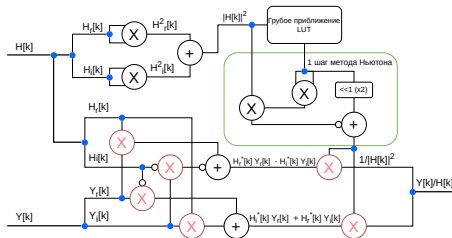
Проблема прямого переноса в параллельную схему

В оригинале — последовательный умножитель: перекодировщик работает по одному такту на код, экономия — от пропуска тактов.

В параллельной схеме пропускать такты нечего — все частичные произведения формируются одновременно.

Перекодировщик пришлось бы ставить в каждую строку:

- рост площади и задержки
- накладные расходы съедают выигрыш от занулений
- арбитраж между строками усложняет схему

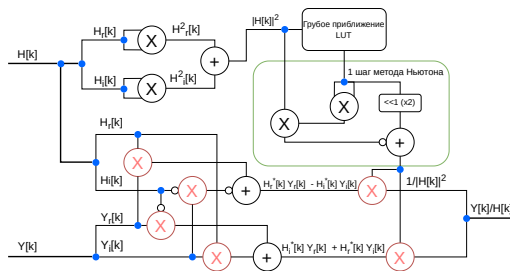


Перекодировщик дублируется в каждой строке

Предлагаемая архитектура: вынесенная перекодировка

Ключевая идея: перекодировщик вынесен **наружу** умножителя.

- В регистре умножителя хранится уже перекодированный вектор (24 бита: two, one, neg)
- Перекодировка считается **один раз** при обновлении коэффициента
- Стоимость амортизируется по N тапам и по времени
- Умножитель — один и тот же RTL для всех режимов (точный / приближённые)



Сверху — базовая схема, снизу — с вынесенной перекодировкой

Три RTL-модуля (Verilog):

- `mult_naive` — прямое $a*b$ (опорная точка)
- `mult_booth` — radix-4 Booth, перекодировщик внутри
- `mult_booth_extrec` — приём готового 24-битного вектора

Перекодировщик — на Python:

- последовательный алгоритм, все 7 паттернов
- генерирует вектор `{two, one, neg}`
- режимы: `exact`, `approx_1..4`

